2. The heap tree is complete, with all the levels that are left-adjusted, i.e. the `i`-th sub-tree has a height at most equal to its siblings on its left (from `0` to `i-1`, `1 < i <= D`)
3. Every node holds the highest priority in the subtree rooted at that node.

Fun fact: it's worth noting that for `D = 1` the heap becomes a sorted array (or a sorted double linked list, in its tree representation). The heap construction will be the insertion sort algorithm, and requires quadratic time. Every other operation requires linear time.

2.6 How to implement a heap

At this point we have a good idea of how a priority queue should be used, and what's their internal representation: it's about time we delve into the details of the implementation for a heap.

Before we go, let's take once again a look at our API:

```
class Heap {
  top()
  peek()
  insert(element, priority)
  remove(element)
```